Agent

Semantic & agentic search

Agent combines multiple search tools to find relevant code across your codebase. You describe what you're looking for in natural language, and Agent picks the right strategy.

Instant Grep

The fastest way to find code is an exact match: a function name, variable, error string, or regex pattern. Agent uses grep automatically when you reference specific symbols.

Cursor ships with Instant Grep, a custom search engine that outperforms `ripgrep` on large codebases. It runs automatically; no configuration needed.

Instant Grep supports full regex and word-boundary matching, so Agent can construct patterns like `import.*PaymentService` or `PaymentFailedError` to trace references across files.

Semantic search

When you don't know the exact name, semantic search finds code by meaning. Ask "where do we handle authentication?" and Agent can locate `middleware/session.ts` even though the word "authentication" never appears in the file.

This works because Cursor indexes your codebase into searchable vectors with a custom embedding model. Research on Cursor's semantic search shows that combining it with grep produces 12.5% higher accuracy answering codebase questions compared to grep alone. The improvement is largest on codebases with 1,000+ files.

How indexing works

Cursor breaks your code into meaningful chunks (functions, classes, logical blocks), converts each chunk into a vector embedding that captures its semantic meaning, and stores the results in a vector database. When you search, your query is converted into a vector using the same model and matched against the stored embeddings.

Indexing begins automatically when you open a workspace. Semantic search becomes available at 80% completion. The index stays current through automatic sync every 5 minutes, processing only changed files.

Change	Action
New files	Added to the index automatically
Modified files	Old embeddings removed, new ones created
Deleted files	Removed from the index

Configuration

Check indexing status or trigger a re-index from **Cursor Settings > Indexing**.

Cursor indexes all files except those in ignore files (`.gitignore` , `.cursorignore`). Ignoring large generated or content files improves search accuracy.

To view indexed file paths: **Cursor Settings > Indexing & Docs > View included files**.

Privacy and security

File paths are encrypted before being sent to Cursor's servers. Code content is never stored in plaintext; it is held in memory during indexing, then discarded. Embeddings are created without storing filenames or source code. When Agent searches, Cursor retrieves the embeddings and decrypts the chunks on the client side.

How Agent combines search tools

Agent picks the right tool based on your prompt:

Prompt style	Tools used	Example
Specific symbol or string	Grep	"Find all files that import `PaymentService`"
Concept or behavior	Semantic search, then grep to fill in details	"How does our app handle failed payments?"
Complex exploration	Multiple searches, file reads, reference following	"Map the data flow from checkout to confirmation email"

You don't choose the tool. Describe what you need and Agent decides. For complex tasks, it chains searches together: semantic search to find entry points, grep to trace references, and file reads to build full context.

Explore subagent

Agent can spawn an Explore subagent that runs in its own context window with a faster model. It executes many parallel searches without bloating the main conversation, returning only the relevant findings.

Agent uses the Explore subagent automatically when it decides a task benefits from broad search. You can also request it directly: "use a subagent to find all the places we validate user input."

This is useful for context management. Searching through many files generates a lot of context. The subagent keeps the main conversation focused by summarizing results instead of dumping raw file contents.

Tips for better search results

- **Start specific, then go broad.** If you know the function name, say it. If you're exploring unfamiliar code, describe the behavior.
- **Explore before changing.** Ask Agent to show existing patterns before asking it to add new ones. This prevents it from creating duplicates or breaking conventions.
- **Reference concrete code.** Prompts like "find all callers of `processOrder`" give Agent an exact target. Prompts like "find the order code" force Agent to guess what you mean.

FAQ